

Characterizing Performance–Energy Trade-offs of Large Language Models in Multi-Request Workflows

MD. MONZURUL AMIN IFATH, Dalhousie University, Canada

ISRAAT HAQUE, Dalhousie University, Canada

Large language models (LLMs) are increasingly deployed in applications forming multi-request workflows like document summarization, search-based copilots, and multi-agent programming. While these workflows unlock richer functionality, they also amplify latency and energy demand during inferences. Existing measurement and benchmarking efforts either focus on assessing performance of LLM inference systems or consider single-request evaluations, overlooking workflow dependencies and cross-request interactions unique to multi-request workflows. Moreover, the energy usage of such interdependent LLM calls is not explored in-depth.

To address these gaps, this paper presents the first systematic characterization of performance–energy trade-offs in multi-request LLM inference. We develop and evaluate four representative workloads that capture sequential, interactive, agentic, and composite patterns common in modern deployments. Using an empirical NVIDIA A100 testbed with state-of-the-art serving systems (vLLM and Parrot), we systematically analyze how key energy knobs (e.g., input-output length, batch size, and GPU power cap) reshape latency, throughput, and component-level (e.g., CPU, GPU, and DRAM) energy use. Our findings reveal that batch size is the most impactful lever, though its benefits are highly workload dependent. While optimal batching benefits workloads with large shared prompts, it is ineffective for sequential summarization and only partially effective for multi-agent coding. GPU power capping provides modest but predictable savings, while output length induces linear energy scaling with limited efficiency gains. We further demonstrate that engine-level optimizations in vLLM (e.g., continuous batching, PagedAttention) maintain higher GPU utilization and efficiency, especially for decode-heavy workloads, while Parrot’s workflow-aware scheduling achieves lower energy consumption under stringent power constraints. These findings offer actionable guidelines for developers and system operators in designing performance- and energy-aware LLM serving systems in emerging multi-request workflows.

CCS Concepts: • **Computer systems organization** → **Cloud computing**; *Client-server architectures*; • **Software and its engineering** → *Power management*.

Additional Key Words and Phrases: Large Language Models; Multi-Request Inference; LLM Serving Systems; Energy Measurement; Sustainable LLM systems; Performance–Energy Trade-offs

ACM Reference Format:

Md. Monzurul Amin Ifath and Israat Haque. 2026. Characterizing Performance–Energy Trade-offs of Large Language Models in Multi-Request Workflows. *Proc. ACM Meas. Anal. Comput. Syst.* 10, 1, Article 7 (March 2026), 26 pages. <https://doi.org/10.1145/3788089>

1 Introduction

Large language models (LLMs) have rapidly become the backbone of modern AI applications, powering productivity assistants, conversational copilots, knowledge-intensive search, and autonomous multi-agent frameworks [12, 16, 73, 78], which are widely adopted in consumer-facing chatbots,

Authors’ Contact Information: Md. Monzurul Amin Ifath, monzurul.amin@dal.ca, Dalhousie University, Halifax, NS, Canada; Israat Haque, israat@dal.ca, Dalhousie University, Halifax, NS, Canada.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2476-1249/2026/3-ART7

<https://doi.org/10.1145/3788089>

domain-specific copilots in programming, healthcare, and education. The key enabler of these applications is LLM *inference*, the process of generating outputs from a pre-trained model given an input prompt. The inference can be single- or multi-request depending on the chain of invocation, such as translating a short text segment (*single-request*) vs. summarizing a long document by invoking a pipeline of segment-by-segment summarization (*multi-request*). The adoption of multi-request workflows is becoming a norm in the LLM application domain, e.g., conversational copilots that iteratively refine search results under safety and context constraints [26, 47], and agentic programming frameworks (e.g., MetaGPT [46], AutoGen [10]) that coordinate specialized roles across architect, coder, and reviewer agents.

Unlike LLM model training, which is computationally intensive but episodic, inference of these multi-request workflows are continuously invoked in respective applications, making inference the dominant factor in resource consumption (e.g., computation and energy) of LLM systems. A plethora of systems [5, 7, 36, 40, 55, 75] have focused on optimizing computational demand of LLMs without accounting on their energy demand and consequences. However, AI inference accounts for 60-70% of AI power use in hyperscale infrastructures [72], with a single chatbot interaction consuming up to 10× more energy than a typical web search [32]. The energy consumption of LLM inference is influenced by a set of well-known knobs, including workload-level parameters such as batch size and input-output length, model-level parallelism strategies (e.g., data, pipeline, or tensor parallelism), and system-level controls such as GPU power capping [22, 45].

Several attempts have been made to characterize the performance (throughput and latency) of single-request inference systems or to explore performance–energy trade-offs over a subset of the available control knobs. In the case of multi-request workflows, some studies focus only on the performance of LLM serving systems, the middleware that connects high-level LLM applications to low-level inference backends. However, multi-request workflows exhibit unique characteristics such as dependencies among LLM calls, heterogeneous prompt structures, and varying complexity levels along critical paths, which reduces the scope of batching, prefix reuse, and latency reduction, respectively. Thus, the performance–energy characterization must consider the interplay between the application layer, the serving system, and the backend (i.e., LLM engine) while incorporating the control knobs at the workload, model, and system level. Thus, comprehensive characterization will enable the design of sustainable LLM systems, which is missing in existing measurements.

We conduct the first comprehensive study of performance–energy trade-offs for multi-request workflow-based LLM inferences. To do so, we consider four representative workloads of document chain summarization, LLM-powered search, multi-agent coding, and composite (see examples in Figure 2). We specifically choose these four workloads because they capture the dominant multi-request patterns observed in practice (e.g., LangChain [2]): sequential pipelines (document summarization), interactive copilots (LLM-powered search), agent-driven directed acyclic graphs (DAGs) (multi-agent coding), and heterogeneous mixes of workloads (composite). Across these workloads, we systematically evaluate three key energy knobs (input-output length, batch size, and GPU power capping) because they have been consistently identified as the most critical levers for shaping the performance–energy trade-off in LLM inference. Batch size directly influences GPU utilization and throughput efficiency, input-output length determines both prefill and decode costs, and GPU power capping provides a practical system-level mechanism to bound energy usage under latency or other service-level objectives (SLOs).

Conducting this study required addressing several practical challenges. Workflow orchestration introduced strong dependencies that complicated cache management and concurrency control. Ensuring accurate and reproducible energy measurement demanded fine-grained monitoring of hardware components (CPU, GPU, and DRAM) while isolating background interference. Serving

system heterogeneity further introduced implementation and deployment variations, requiring careful tuning and validation. To mitigate these challenges, we explicitly modeled dependencies within workflow definitions and validated execution order, fixed concurrency levels to maintain workload consistency, and standardized prompt templates across runs. Each serving system was containerized to ensure isolation. Under stable thermal conditions, we utilized Zeus [34] to measure component-level energy consumption, for repeated trials to ensure dependability. All workloads were deployed and evaluated on a controlled NVIDIA A100 testbed for consistency and reproducibility.

The evaluation first investigates which knobs are most influential on workload efficiency, how the most impactful knob shapes performance–energy trade-offs across applications, and how different serving system designs behave under the most energy-intensive workload. Our findings highlight that *batch size* is the most impactful knob, reduces energy per token by up to 2.6× for LLM-based search workloads, but negatively impacts sequential summarization when dependencies dominate. The output length exhibits near-linear energy scaling with limited efficiency gains, while GPU power capping offers modest but predictable energy savings at the expense of throughput.

Finally, we compare two widely adopted serving systems, vLLM [36] and Parrot [41], under the most energy-intensive workload (document chain summarization). We focus on vLLM and Parrot as they embody two contrasting design philosophies. vLLM embodies high-throughput, engine-level optimizations, while Parrot adopts workflow-aware scheduling through semantic variables that capture inter-request dependencies. Our results show that vLLM’s low-level optimizations sustain higher GPU utilization, leading to consistently lower energy consumption. In contrast, Parrot achieves relative gains only under strict power constraints, leveraging its workflow-aware scheduling to balance CPU-GPU activity. For example, in the document chain summarization with batch size 16, vLLM reduced GPU stall time and achieved consistently better utilization, offering up to 28% lower energy consumption than Parrot. We summarize our main contributions as follows.

- We develop and evaluate four multi-request workloads under a controlled NVIDIA A100-based testbed with fine-grained CPU, GPU, and DRAM energy monitoring.
- We conduct the first comprehensive analysis of performance–energy trade-offs in workflow-based LLM inference, focusing on three critical energy knobs (input-output length, batch size, and GPU power capping).
- We compare two serving systems, vLLM and Parrot, to demonstrate how engine-level optimizations contrast with workflow-aware scheduling in shaping energy efficiency, particularly under high-load.
- Our findings highlight that (i) batch size is the most impactful but workload-dependent knob, reducing energy per token by up to 2.6× in search while degrading sequential summarization; (ii) output length induces proportional energy scaling with limited efficiency gains; and (iii) GPU power capping provides modest, yet predictable, energy savings.

The remainder of this paper is structured as follows. Section 2 introduces the foundations of LLM serving systems and energy control knobs. Section 3 surveys related work. Section 4 describes our experimental setup and measurement methodology. Section 5 presents empirical results addressing three research questions. We discuss our findings further in Section 6. We make concluding remarks in Section 7.

2 Background

This section starts with the architectural components of LLM inference (see Figure 1), where we first describe the LLM application layer, which captures workflow-level properties and orchestration of multi-request applications. Next, we examine the LLM serving layer, which connects applications to low-level inference backends and manages system-level optimizations. We conclude with an

overview of energy measurement and control knobs that frame our study of performance–energy trade-offs in workflow-based LLM serving.

LLM Inference: Modern LLMs primarily adopt the transformer architecture [70], which comes in three main variants: encoder-only, decoder-only, and encoder-decoder [49]. For generative tasks, decoder-only architectures (e.g., GPT [13], LLaMA [68]) are the most common, which produce output tokens sequentially by conditioning input tokens and contextual representations embedded in the model parameters. This sequential token generation (aka *auto-regressive*) processes one token at a time.

LLM inference typically consists of two distinct stages: *prefill* and *generate (decode)*. In the prefill stage, the input prompt is tokenized and the model computes the attention relationships among these tokens. In the generate stage, the model predicts the next token iteratively, each time leveraging the previously generated tokens. To avoid recomputing attention for past tokens [75], the intermediate key-value representations are stored in a dedicated memory structure called *KV cache*. The prefill stage exhibits high parallelizability, lead to computation bottleneck (i.e., being compute-bound), while the auto-regressive decoding stage is primarily limited by the memory-bandwidth due to repeated access to model weights and the KV cache [55].

LLM Application Layer: LLM-based applications often extend beyond a single model invocation, forming multi-step workflows where multiple LLM calls are connected through well-defined control and data dependencies [44]. For example, long document summarization uses chain style pipelines, LLM-based search iteratively reformulates and synthesizes answers, and multi-turn planning-execution loops in multi-agent¹ programming paradigms. In such workflows, the output of one stage often becomes the input to another and the overall application latency is determined by the critical path (the longest chain of dependent LLM calls determines the end-to-end completion of a workflow) across these stages. Capturing and exposing these dependencies allows LLMs to make informed scheduling decisions that prioritize tasks that unlock downstream progress [41]. Orchestration frameworks (e.g., LangChain [2], LlamaIndex [3]) provide higher-level abstraction to compose these workflows. They expose APIs for chaining model calls, integrating external tools, and coordinating multi-agent interactions, thereby generating the structured execution graphs. These frameworks form the core of the LLM *Application Layer* and act as the glue between applications and serving systems, ensuring that workflow semantics are preserved while delegating low-level execution to serving systems in the later layer.

LLM Serving Layer: In modern LLM-based deployments, the serving layer functions as the critical middleware connecting high-level applications (e.g., agents, multi-request workflows) to low-level inference backends (e.g., GPU runtimes, quantized engines). A well-designed serving system must take care of scheduling and multiplexing requests across constrained hardware, mediating latency-throughput trade-offs, and incorporating orchestration decisions from the application layer [5, 38, 67]. In practice, serving systems are responsible for batching, queuing, resource partitioning, model routing, KV cache reusing, and in advanced cases, application-aware scheduling or DAG-level

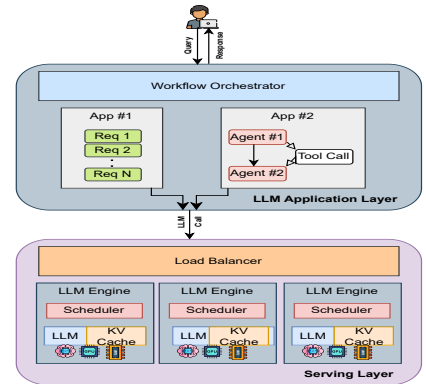


Fig. 1. Architectural layers of LLM inference, spanning the LLM application layer (top) and the LLM serving layer (bottom).

¹An LLM agent refers to an autonomous module driven by an LLM that performs a specific role (e.g., planning, coding, or reviewing) and communicates through structured prompts and responses within a coordinated workflow [73].

optimization. Example includes vLLM [52], Parrot[48], llama.cpp [25], Ollama [51], Huggingface Text Generation Inference (TGI) [30], SGLang [60].

vLLM is a high-performance inference engine and serving system focused on memory efficiency and throughput. Its key innovation, *PagedAttention* [36], treats the KV cache as a paged memory space, dynamically allocating and evicting pages to reduce fragmentation under heterogeneous request lengths. Combined with continuous batching (*i.e.*, dynamically admits new requests into in-progress batches) and memory reuse, vLLM achieves high throughput while maintaining competitive latency. In contrast, Parrot introduces the *Semantic Variable* abstraction to expose application-level structure to the serving system. By annotating inputs and outputs as semantic variables, applications define a DAG across dependent calls. This information enables Parrot to co-schedule dependent requests, eliminate redundant prompt prefixes, and optimize directly for end-to-end latency rather than individual request times.

SGLang combines a structured generation domain specific language (DSL) with a high-performance runtime, leveraging techniques such as RadixAttention for prefix reuse, speculative decoding, and lightweight scheduling, targeting a balance between throughput and workflow flexibility. Ollama, built on top of llama.cpp, provides a user-friendly local server with quantization and model management, suitable for personal or small-scale deployments but limited in concurrency and batching. Finally, Hugging Face TGI provides a production-ready serving layer with support for continuous batching, multi-GPU scaling, and optimized kernels, but does not incorporate fine-grained application-level semantics into its scheduling decisions. A summary of these systems is shown in Table 1, positioning their abstractions, focus, and limitations.

System	API / Abstraction	Focus (Latency vs Throughput)	Limitations
vLLM	OpenAI-style text completion API	High throughput with modest latency	Performance in multi-call workflows is under-explored
Parrot	Semantic Variables/ DAG API	End-to-end latency in multi-step workflows	Requires application annotations; not tuned for pure throughput
SGLang	Structured generation DSL + runtime	Balanced workflow-aware performance	Newer system; steeper programming model
Ollama	Local server wrapping over llama.cpp	Low-latency, small-scale use	Limited batching and scale-out
llama.cpp	Minimal HTTP/ OpenAI-style API	Simple, edge deployment	Minimal scheduling; low throughput under load
HuggingFace TGI	REST/ gRPC text-gen API	Scalable throughput with reasonable latency	Heavyweight system; no workflow awareness

Table 1. Comparison of representative LLM serving systems.

Given this landscape, our study focuses on vLLM and Parrot. vLLM serves as the benchmark for engine-level throughput and memory optimizations, establishing the baseline efficiency of single-request inference. Parrot, by contrast, represents the application-aware paradigm, where exposing cross-call dependencies enables more holistic optimization of workflow performance. Together, these systems allow us to examine both the benefits of inference-layer optimization and the additional gains unlocked by workflow-aware scheduling.

Energy Measurement: The influencing factors in the energy consumption of LLM inference includes spanning model-, workload-, system-, and platform-level categories. Model-level knobs include model size, architecture, parallelism, and optimization techniques [9, 17, 58, 65], while

workload-level knobs cover batch size and input-output length [64, 71, 74]. System-level controls include the number of GPU instances, and GPU power capping [54, 74], while platform-level factors encompass the choice of deep learning inference frameworks (e.g., PyTorch, TensorFlow) [24]. Among these, three knobs at the workload and system levels have been consistently highlighted as critical for controlling the performance–energy trade-off [58, 64, 71]. First, the LLM request input-output length directly impacts computational cost and memory access, thereby shaping energy usage [45, 71]. Second, batching plays a decisive role in GPU utilization and throughput efficiency, with larger batch sizes improving amortization of compute overheads but increasing latency sensitivity [58, 64]. Third, GPU power capping provides a practical mechanism to bound energy consumption under varying SLOs [9, 35, 58].

Input-output Length: The input length in LLM inference refers to the number of tokens present in a submitted request. On the contrary, the output length is the number of tokens generated by the model in response to that request. Increased input length necessitates greater GPU parallelism, leading to extended prefill latency; conversely, longer output sequences induce more auto-regressive iterations, which consequently increase the decode phase. Typically, the output length is explicitly capped by the application, and the inference engine halts generation upon reaching this predefined token limit. This design allows precise control over the response size and latency, which is particularly important for performance-sensitive workloads.

Batch Size: Batch size is another workload-level parameter in LLM serving refers to the number of concurrent requests grouped and processed together in a single forward pass of the model. Batching improves GPU utilization and inference throughput by reducing memory access and computation costs across multiple requests. However, this comes at the cost of increased per-request latency due to queuing and batching delays. The optimal batch size depends on the application’s tolerance to latency and the nature of the workload.

GPU Power Capping: Power capping provides a system controllable hardware-level parameter to control the energy footprint of LLM inference. Modern GPUs are typically configured to operate at their peak power to maximize throughput and minimize latency. However, for use cases that are not latency-critical (e.g., offline summarization or scheduled analytics tasks), operating at full power is unnecessary and may lead to energy inefficiency. Power capping enforces an upper bound on GPU power draw, enabling energy savings with potentially acceptable trade-offs in performance. Intervention to enforce GPU power or frequency limits introduces measurable, non-negligible system overhead [65].

3 Related Work

Related work in this section is grouped into two categories: (i) serving systems and orchestration frameworks that enable efficient execution of multi-request LLM applications and (ii) empirical studies and measurement methodologies that quantify inference energy consumption in practical deployments.

LLM Orchestrator Frameworks and Serving Systems: Modern orchestration frameworks provide developers with primitives to manage application’s control flow while deciding when to trigger multi-request workflows, invoke tools, or ask for human input. To handle complex inter-agent communication and tool execution patterns within LLM-based applications, frameworks such as LangChain [2], LlamaIndex [3], and AutoGen [10] have emerged. These frameworks enable developers to compose multi-step workflows by chaining LLM calls, integrating external tools (e.g., databases, APIs), and managing agent interactions through declarative or graph-based specifications [11, 59, 77]. Kairos [15] further targets multi-agent orchestration under high concurrency, using workflow- and memory-aware scheduling. These frameworks primarily focus on responsiveness

and scalability while efficiently utilizing underlying system resources. They achieve this by exposing high-level abstractions for client-side workflow specification, which simplify application composition and interaction logic. Some frameworks also consider fairness, *i.e.*, ensuring balanced scheduling across agents or users [14].

These orchestration frameworks interact with the serving systems that connect these frameworks to LLM engines. Recent studies have systematically benchmarked serving systems, exposing key performance trade-offs. LLM-Inference-Bench [17] is a benchmarking suite for various accelerators and engines such as vLLM, llama.cpp, and TensorRT-LLM, revealing heterogeneous scaling patterns across prompt lengths, model sizes, and batching. DeepSpeed-FastGen [27] improves the performance of vLLM while studies in [4, 36, 75] focus on other notable serving systems like HuggingFace TGI, Ollama, and SGLang. Agrawal *et al.* [4] propose a serving system with fine-grained batching mechanisms and better performance.

However, these serving systems are single-request-centric and optimize throughput and memory efficiency without exploiting cross-request relationships in multi-step workflows. Parrot [41] fills that gap by introducing semantic variables that expose dependency graphs for co-scheduling and cache reuse, directly optimizing workflow-level performance. Similarly, Autellix [44] adopts system-level scheduling policies that treat LLM agents as interdependent execution units. Together, Parrot and Autellix mark a shift from isolated request execution toward workflow-aware serving, where end-to-end efficiency (rather than per-call latency) becomes the optimization objective.

LLM Energy Consumption: Several studies have examined the energy consumption and optimization of deep learning and LLM training [23, 24, 57, 74]. García-Martín *et al.* [23] surveyed energy estimation techniques in machine learning, providing a taxonomy of power models and discussing their applicability to both training and inference. Georgiou *et al.* [24] compared the energy costs of popular deep learning frameworks (e.g., PyTorch, TensorFlow) during training, revealing notable framework-dependent variations. FECoM [57], a fine-grained measurement framework for TensorFlow APIs, enables API-level profiling and demonstrates how parameter size and execution time influence energy consumption. These works emphasize that accurate estimation, framework-level profiling, and fine-grained measurement are key enablers for energy-aware deep learning practices. Zeus [74] developed an optimization system based on fine-grained measurements that balances performance and energy efficiency by automatically finding optimal GPU-level and job configurations for recurring deep neural network training.

Complementing training-oriented studies, a growing body of work now focuses on energy consumption during inference, which dominates cumulative usage in deployed AI systems. For example, Lahmer *et al.* [37] and Tu *et al.* [69] evaluated the energy usage of deep learning models on edge devices such as NVIDIA Jetson boards. Sobhani *et al.* [63] conducted a systematic characterization of performance-energy trade-offs across various edge platforms (e.g., Raspberry Pi, Google Coral) for machine learning, deep learning, and LLM inference. Their analysis reveal that hardware selection, lightweight frameworks, and inference parameters collectively influence both performance and energy efficiency.

In parallel, several software-based energy meters have emerged to enable reproducible measurement and analysis of inference workloads. Argerich and Patiño-Martínez [9] propose *EnergyMeter*, a profiler capable of attributing CPU, GPU, memory, and storage energy consumption during inference. Similarly, Zeus [74], also supports accurate measurement of inference-time energy consumption across hardware components. Huang *et al.* [29] advance this direction through *WattsOnAI*, which unifies energy, power, and carbon metrics into a visualization toolkit for comprehensive sustainability analysis. *CodeCarbon* [19] has further popularized lightweight carbon accounting by integrating runtime energy estimation directly into ML pipelines.

Finally, energy knob-based characterization and benchmarking studies systematically analyze how LLM inference configuration parameters impact the inference energy. Samsi *et al.* [58] provide one of the first cross-platform baselines, profiling the energy consumption of LLM inference across various hardware and workloads. Stojković *et al.* [64] identify key control parameters such as input-output length, batching, and GPU frequency scaling, which govern the performance-energy balance. Wilkins *et al.* [71] further develop workload-based energy models that capture the joint impact of input-output lengths on runtime and energy. Maliakel *et al.* [45] empirically explore the interplay between task type, sequence length, and clock frequency, while Argerich and Patiño-Martínez [9] demonstrated the effects of batch size, quantization, and model architecture on efficiency. Rajput *et al.* [56] show that orthogonal combinations of model-, system-, and inference-level knobs can support cascading energy savings.

Comparison with existing work: Prior research on LLM orchestration and serving systems has primarily optimized for throughput, latency, and scalability, focusing on responsiveness and resource utilization rather than energy efficiency. In contrast, studies on LLM inference energy consumption focused on accurate measurement and energy knob-based characterization, but evaluations largely remain at the single-request inference level, overlooking workflow dependencies and cross-request interactions that dominate emerging LLM applications.

Our work bridges these gaps by conducting the first comprehensive characterization of performance-energy trade-offs in multi-request LLM workflows. We systematically evaluate multiple energy knobs in representative multi-request workflows and quantify how engine-level optimizations, workflow-aware schedulers behave under interdependent requests. This unified perspective reveals new sustainability challenges such as limited batching efficiency, reduced cache reuse, and compounded critical-path latencies that are absent in single-request studies, establishing the foundation for energy-aware LLM serving and orchestration.

4 Experimental Setup and Implementation

This section constructs a controlled deployment environment that integrates hardware, serving systems, measurement tools, and representative workloads. We begin by detailing the hardware and software stack of our testbed, followed by the deployed serving systems and the energy measurement framework. We then describe the four representative multi-request workloads that form the basis of our experiments. Together, these components provide the methodological foundation for our results in Section 5, enabling a systematic assessment of how key energy knobs influence sustainability in workflow driven LLMs.

4.1 Setup

Testbed Setup: To ensure consistent and reliable results, we conducted our experiments on a dedicated server (Supermicro SYS-740GP-TRT). The server is equipped with two 12-core Intel Xeon 4310 CPUs (2.1 GHz, 18 MB cache) and one NVIDIA Ampere A100 GPU [18] with 40 GB of HBM2 memory. It has 128 GB of DDR4 2933 MHz RAM and a total of 26 TB of storage capacity. This single-GPU setup allows us to isolate workload-level behaviors and component-wise energy dynamics without interference from communication or synchronization overheads common in distributed settings. This isolation is essential to establish a reproducible performance-energy baseline before extending to multi-GPU scenarios. Power is supplied by redundant 2200W (80 PLUS Titanium) high-efficiency power supply units. The software stack includes CUDA 12.6, cuDNN 9.1.0, and Ubuntu 24.04.2 LTS with kernel 6.5.0-27-generic.

Serving Systems: We evaluate our workloads on two serving systems, vLLM [52] and Parrot [41], which embody distinct design philosophies. vLLM is widely adopted for high-throughput inference and integrates advanced optimizations such as FlashAttention [20], continuous batching

[75], and PagedAttention [36] to improve GPU utilization and reduce memory fragmentation. We use vLLM V1 engine [53] (version 0.9.1) and access vLLM through its Python bindings, which implement an OpenAI-compatible interface for LLM engine interaction. In contrast, Parrot introduces semantic variables to expose inter-request dependencies, enabling workflow-aware scheduling optimizations that are particularly suited for multi-request LLM applications. This design allows Parrot to coordinate scheduling across dependent calls, targeting lower end-to-end latency for structured workloads. Parrot exposes its query submission and configuration endpoints through a FastAPI-based interface [1]. Together, these systems provide a representative comparison between request-level and workflow-level serving designs under performance–energy tradeoffs. All evaluations are conducted using isolated Docker containers for vLLM and Parrot.

Large Language Model: The evaluation is based on an open-source implementation of the Llama-2 model (with 7 billion parameters), available via request from Meta, a pre-trained variant from the Llama family of decoder-only models [68]. Decoder-only transformers have become the dominant architecture for generative LLM deployments due to their simplicity and efficiency in autoregressive token generation. Llama-2-7B, in particular, is optimized for instruction-following tasks and has been widely adopted in research as a representative open-source alternative to commercial models [76]. The model supports a context size of 4096 tokens and is trained using conversations collected from ShareGPT [61]. We use the HuggingFace implementation [31] of Llama-2-7B with PyTorch and the Transformers library, together with the default *llama-tokenizer* as the tokenizer backend. For model configuration, we set the decoding parameters at a temperature of 0.7 and a top-p value of 1.0. These settings provide a balance between diversity and determinism in generation. Moreover, the settings are consistent with the experimental setups used in both serving-system research and sustainability benchmarking [64].

Energy Measurement: A key enabler for energy measurement is selecting effective tools. We choose Zeus [74] as our energy measurement framework for LLM inference. Zeus is explicitly designed for deep learning workloads, providing both accurate energy consumption monitoring and optimization capabilities. It integrates tools and libraries such as *nvidia-smi* [50], *amdsmi* [8], and *RAPL* [21], to capture fine-grained, component-wise energy across CPU, GPU, and DRAM. This unified interface enables reproducible measurement without requiring hardware modifications, and supports a wide range of accelerators, including NVIDIA GPUs, AMD GPUs, Apple Silicon, and Jetson-class embedded platforms. Beyond measurement, Zeus also includes support for energy optimization workflows, making it an appropriate foundation for sustainability-oriented inference studies. It is widely adopted, actively maintained, and well received by the community, ensuring reliability and reproducibility for our evaluation setup.

Zeus reports windowed average energy consumption at one-second granularity, which is well suited for capturing the dynamics of multi-request LLM workloads [56]. All measurements are conducted under stable thermal conditions with active cooling and isolating the inference process to eliminate background interference so that only the energy consumed during model execution is captured. Listing 1 presents a unified, workload-agnostic energy measurement framework that standardizes execution and metric collection across all evaluated multi-request workflows (see workload description in Section 4.2). In the listing, *LLMBackend* denotes an abstract serving-system interface that encapsulates the underlying LLM inference engine. In our evaluation, this abstraction is instantiated using vLLM or Parrot, enabling a uniform execution and measurement pipeline across different serving-system designs.

Challenges: While constructing this evaluation environment, we encounter several practical challenges that required careful mitigation. First, workflow orchestrations introduce strong inter-request dependencies, making it difficult to isolate requests for batching and complicating cache management. We address this by explicitly modeling dependencies in workflow definitions and

validating execution order to ensure consistency across trials. Second, concurrency control is non-trivial as heterogeneous request arrivals in multi-agent and composite workloads created variability in queueing and scheduling, which we mitigate by fixing user concurrency levels and aligning prompt templates across runs. Third, accurate energy measurement pose challenges due to noise from background processes and component-level drift. To address this, we isolate inference processes within dedicated Docker containers, stabilize thermal conditions with active cooling, and repeat each experiment ten times to capture statistically significant averages. Finally, heterogeneity across serving systems required careful configuration. For instance, vLLM's continuous batching and Parrot's semantic variables expose different tuning knobs, which we coordinate by standardizing decoding parameters, maximum sequence lengths, and energy measurement intervals. Following these mitigation steps, we ensure that our evaluation pipeline remained reproducible, fair, and representative of real-world LLM deployments.

Listing 1. Unified workload-agnostic pseudo-code for multi-request LLM workloads with energy measurement.

```

1  from zeus.monitor import ZeusMonitor
2
3  # ... (imports, configuration, environment setup) ...
4
5  # 1) Initialize energy monitor and LLM backend
6  zeus_monitor = ZeusMonitor()
7  llm_backend = LLMBackend(...) # abstract serving-system interface
8  gen_cfg = {"model": "...", "max_tokens": "...", "temperature": 0.7}
9
10 # 2) Define generic LLM inference interface
11 def llm_request(prompt, **gen):
12     """
13     Abstract LLM invocation.
14     The backend encapsulates the serving system (e.g., vLLM, Parrot)
15     and returns generated text along with token usage statistics.
16     """
17     text, token_stats = llm_backend.generate(prompt, **gen)
18     return text, token_stats
19
20 # 3) Define workload-specific prompt generation
21 def get_workload(workload, *, chunks=None, queries=None, system_prompt=""):
22     """Returns an iterable of prompts based on workload type."""
23     chunks = chunks or []
24     queries = queries or []
25
26     workloads = {
27         "summarize": [f"Summarize {c}" for c in chunks],
28         "search": [f"{system_prompt} Query: {q}" for q in queries],
29         "agentic": [
30             "[Architect] Design...",
31             "[Engineer] Implement...",
32             "[Reviewer] Review...",
33             "[Reviser] Revise..."
34         ],
35         # Composite workload: heterogeneous sub-workloads
36         "composite": (
37             [f"Summarize {c}" for c in chunks] +
38             [f"{system_prompt} Query: {q}" for q in queries]
39         ),

```

```

40     }
41     return workloads.get(workload, [])
42
43 # 4) Execute workload with energy and performance accounting
44 def run_workload(workload, *, chunks=None, queries=None, system_prompt=""):
45     # Begin workflow-level energy window
46     zeus_monitor.begin_window(workload)
47
48     total_out_tokens = 0
49     # Workflow-level latency is captured by the measurement window
50     for prompt in get_workload(
51         workload,
52         chunks=chunks,
53         queries=queries,
54         system_prompt=system_prompt
55     ):
56         _, token_stats = llm_request(prompt, **gen_cfg)
57         total_out_tokens += token_stats.get("output_tokens", 0)
58
59     mes = zeus_monitor.end_window(workload)
60     return mes, total_out_tokens
61
62 # 5) Report metrics
63 mes, out_tokens = run_workload(
64     workload="workload_name",
65     chunks=[...],          # used by summarize/composite
66     queries=[...],         # used by search/composite
67     system_prompt="..." # used by search/composite
68 )
69 report = {
70     "Energy": {
71         "E_cpu (J)": mes.cpu,
72         "E_gpu (J)": mes.gpu,
73         "E_dram (J)": mes.dram,
74         "E_total (J)": mes.total
75     },
76     "Latency": {
77         "T_end_to_end (s)": mes.time_s
78     },
79     "Throughput": {
80         "Tokens/s": out_tokens / mes.time_s
81     },
82     "Metadata": {
83         "Workload": workload,
84         "Model": gen_cfg["model"],
85         "Gen params": gen_cfg,
86         "Backend": "...",
87         "Runtime env": "..."
88     }
89 }

```

4.2 Workloads Specification

We carefully select four representative multi-request LLM workflows that reflect both practical deployment scenarios and stress-testing conditions in data center environments [15, 44, 48]. Workload implementations largely follow LangChain [2], a widely used orchestration framework for developing and deploying LLM-based applications.

Document Chain Summarization: Document chain summarization represents a key workload for knowledge-intensive domains such as scientific publishing, legal services, and government reporting, where single-pass summarization is infeasible due to the limited context length of current LLMs [16, 43]. Each workflow instance typically invokes 2-40 LLM requests. To manage this multi-request dependency, the workload implements a sequential abstractive summarization process tailored for long-form documents (Figure 2a). The summarization process iteratively works on the semantically coherent segments (also known as chunks) of a source document. The initial summary is produced from the first segment while subsequent segments are integrated through a refinement loop based on their importance to the summary. A token budget is applied to bound the output of the evolving summary, ensuring concise and cohesive coverage of the source. In the system perspective, document chain summarization stresses serving infrastructures with strong inter-request dependencies, since the output of one LLM call is passed as part of the input to the next. As each call must generate an abstractive summary before proceeding to the next, the workload is inherently decode-heavy and sensitive to per-token generation cost. These sequential dependencies limit batching opportunities, worsen latency accumulation and introduce challenges in cache reuse and state management.

We select a large collection of academic Arxiv papers [39] that represent long-form inputs to evaluate the effectiveness and efficiency of chain-style summarization. Given the substantial length of many Arxiv articles, we impose a cap on the number of input tokens processed from each document, which ensures a uniform and controlled input size across evaluations, avoiding resource bottlenecks while still preserving sufficient semantic content for meaningful summarization. Since summarization processes large chunked inputs, it directly amplifies prefill computation, making it the only workload where input length significantly impacts energy–performance trade-offs. We therefore evaluate input length only for this workload to capture this distinctive effect.

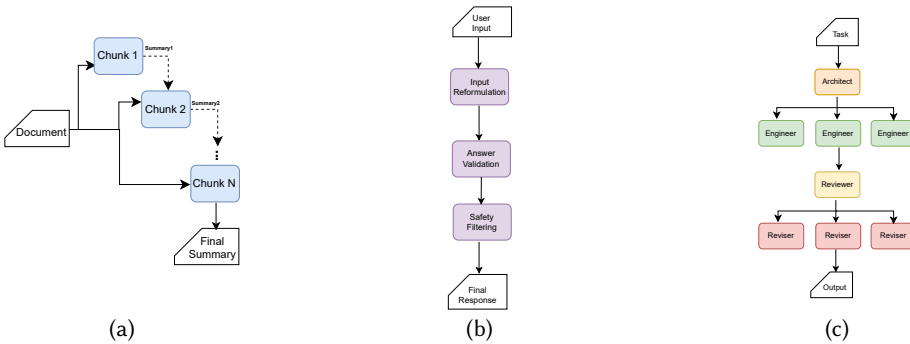


Fig. 2. Representative multi-request LLM workflows in (a) Document chain summarization, (b) LLM-powered search, and (c) Multi-agent coding workload.

LLM-powered Search: This workload models an assistant-style application inspired by productivity copilots that integrate LLMs into search and office environments such as Bing Chat [47] and Google Gemini [26]. The defining characteristic of the workload is the use of a large system prompt that encodes extensive instructions, safety guidelines, and contextual knowledge before any user

query is processed (Figure 2b). Each interaction then appends the user’s query to this persistent prompt. Overall, each user session typically triggers 2-8 LLM calls to generate a contextually grounded response. From an application perspective, this reflects real-world deployments, where continuous context and safety constraints must be preserved while serving diverse user requests. From a systems perspective, the workload is dominated by heavy prefill costs due to the long system prompt, followed by generation steps that require maintaining responsiveness under interactive conditions. However, unlike the document chain summarization workload, LLM-powered search maintains a static shared prefix across all requests. Consequently, its prefill cost is amortized through prefix caching, making input length variations less impactful as an energy knob and not measured separately. Overall, this combination stresses serving infrastructures by increasing memory pressure and latency in the prefill phase, while still demanding efficient throughput during response generation.

We initialize this workload with a system prompt of approximately 3000 tokens, encoding role specifications, safety guidelines, domain-specific instructions, and few-shot exemplars. This static prefix is preserved across all user interactions, reflecting real-world deployments where safety and consistency must be enforced across sessions. On top of this shared context, we synthesize a workload of 64 requests drawn from a diverse query set (covering topics such as AI, IoT, renewable energy, and biology), each appended to the static prompt.

Multi-Agent Coding: This workload captures collaborative software development orchestrated through multiple specialized LLM-driven agents. As in Figure 2c, the task is framed as building a functional program (e.g., a simple snake game) by decomposing the workflow into distinct roles such as architect, engineer, reviewer, and reviser. Each role contributes iteratively to the codebase where the architect specifies the system design and APIs, the programmers implement the code, the reviewers provide quality feedback, and the revisers integrate revisions based on review comments. This creates a pipeline of dependent LLM calls where intermediate outputs (i.e., design specifications, code implementations, review feedback) are passed between agents to progressively refine the software artifact. The workflow typically comprises 15 to 20 LLM calls across all the agents. From a systems perspective, the workload stresses serving infrastructures with dynamic, role-driven workflows that consist of multiple interdependent requests. The execution graph exhibits a directed acyclic graph (DAG) where downstream tasks cannot proceed until upstream agents complete, limiting opportunities for batching and introducing sensitivity to latency accumulation. To align with emerging agentic frameworks, our implementation follows the MetaGPT paradigm [28], which provides structured role assignments for multi-agent collaboration.

In our implementation, each agent role is expressed as a prompt template, and their interactions form a fixed DAG of dependent calls. Input and output token limits are capped per role (e.g., ~1000 tokens for coding, ~500 for review/revision) to ensure controllable execution while preserving semantic fidelity. This setup reflects practical multi-agent deployments and provides consistent measurement across repeated trials [14].

Composite Workload: This workload integrates heterogeneous applications to capture the diversity of LLM usage in practical deployments. By doing so, it also mimics datacenter multi-tenant scenarios where diverse services share the same GPU pool, allowing us to study interference effects and resource contention patterns. For implementation, we combine the chain summarization workload with the LLM-powered search workload and execute them concurrently on the same vLLM/Parrot deployment (with energy monitoring) to capture realistic multi-application contention. While the LLM-powered search application is characterized by long static system prompts and short, interactive user queries, the chain summarization task involves multi-stage iterative refinement over long-form documents with strong inter-request dependencies. Running these workloads concurrently creates a composite scenario where distinct execution patterns coexist. Prefix-heavy,

latency-sensitive copilot queries overlap with sequential, stateful summarization pipelines where each workflow instance issues a variable number of LLM calls drawn from both the summarization and search pipelines. From a systems perspective, such mixed settings stress serving infrastructures with competing demands on batching efficiency, cache reuse, and scheduling fairness. The heterogeneity amplifies contention for GPU resources, as workloads vary in prompt length, output size, and interactivity.

4.3 Metrics

We evaluate the following metrics, repeating each experiment ten times and reporting the mean with a 95% confidence interval.

Workflow Mean Latency: For a given workflow, the end-to-end latency is measured as the elapsed time between the submission of the first request and the completion of the final request in the workflow. The overall metric is reported as the arithmetic mean across all runs in the experiment. Unlike per-request metrics such as Time-to-First-Token (TFTT) or Time-per-Output-Token (TPOT), workflow-level latency reflects the holistic performance of multi-request applications, making it a more representative measure for complex LLM-based workloads [44, 48], which we report in seconds. For brevity, we refer this metric as latency throughout the rest of the paper.

Throughput: We report throughput in tokens per second (tokens/s), which we compute over the generation phase only, as this phase dominates runtime and energy and best reflects steady-state efficiency. Throughput quantifies the total number of tokens generated per unit time across completed requests, capturing how effectively the system sustains token production. It serves as a key metric for analyzing performance-energy trade-offs in LLM inference, complementing latency-based measures [17, 58].

Workflow Mean Total Energy: We measure the total energy consumed by each workflow by aggregating per-component power traces (CPU, GPU, and DRAM) over the workflow's execution interval. The total energy for each run is computed by summing the sampled power readings across all components and time windows, using RAPL for CPU/DRAM and NVML for NVIDIA GPU monitoring. The workflow mean total energy is then obtained as the average across all experiment runs. We report values in joules (J). This metric captures the full-system energy cost across major hardware components, providing a direct measure of sustainability impact beyond what latency or throughput alone can represent [9, 64]. For brevity, we would refer to this metric as total energy.

Energy per Token: We measure energy efficiency using the metric of energy per token, defined as the ratio between the total energy consumed during the workflow execution and the total number of output tokens generated. This metric normalizes energy consumption by the volume of useful work performed, capturing how efficiently the serving system converts energy into generated tokens [58]. We report results in joules per token (J/token).

5 Results

This section presents the evaluation results by answering the following research questions.

- RQ1: What workload- and system-level energy knob(s) most significantly influence the performance and efficiency of multi-request LLM workloads?
- RQ2: How does the most influential energy knob (identified in RQ1) shape the energy-performance trade-offs across representative multi-request LLM workloads?
- RQ3: For the most energy-demanding workload, how do state-of-the-art serving systems (vLLM and Parrot) compare in terms of performance and energy efficiency?

5.0.1 RQ1: Impact of energy knobs in multi-request LLM applications. To answer the first research question, we begin by considering three representative knobs that are widely exposed in serving

systems: output length, batch size, and GPU power capping. These knobs reflect common configuration levers available to practitioners and they allow us to explore how different workload- and system-level choices affect both performance and energy consumption. We evaluate their impact using the *LLM-powered search* workload from our previously defined set of representative applications. We select this workload to answer RQ1 because search-oriented copilots have become one of the most common entry points for general users to interact with large language models. Consequently, even modest per-query energy costs can scale to substantial aggregated consumption, making performance–energy trade-offs in this workload especially impactful in practice [33, 71].

From an implementation standpoint, we set `enable_prefix_caching=True` to share static prefixes across concurrent requests. Responses are capped at a maximum of 800 output tokens, aligning with typical constraints in interactive copilots where latency and readability are prioritized over arbitrarily long generations. To capture concurrency effects, the workload is executed with 8 simultaneous users, each issuing separate independent queries in parallel. This design stresses both the prefill and generation phases of LLM inference. Prefill latency is dominated by the long static prompt, while generation must sustain responsiveness under concurrent load. We conduct this experiment using the vLLM serving system as it has emerged as the de facto in the domain of open-source LLM serving due to its high-throughput design, advanced batching strategies, and efficient memory management mechanisms. Figure 3a-3f depict the effect of varying one of the knobs (output length, batch size, and GPU power cap) while keeping the other two fixed at their default settings, illustrating the corresponding latency–energy or throughput–energy relationships. Here, we report normalized values for latency, throughput, and energy to facilitate consistent scaling and comparability across knobs.

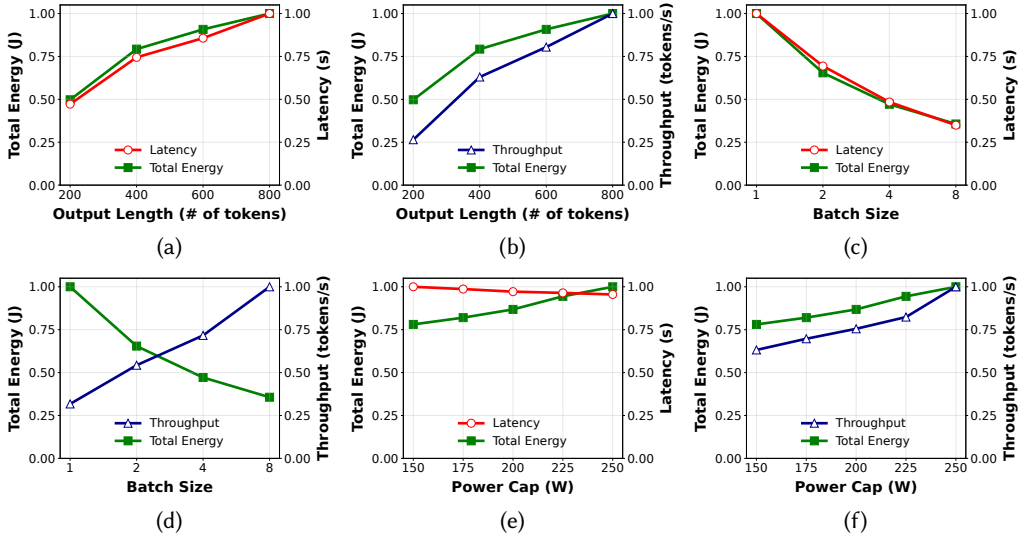


Fig. 3. RQ1 results on the *LLM-powered search* workload. Panels (a-b) sweep output length, (c-d) sweep batch size, and (e-f) sweep GPU power cap, with all other knobs fixed. Each panel uses dual Y-axes with normalized total energy in left vs. normalized performance (latency/throughput) in right Y axis.

Figure 3a shows the effect of varying output length (200-800 tokens) where increasing the output length significantly raises latency and energy consumption. Increasing the output length leads to a near-linear rise in both workflow latency and energy consumption. This trend is expected since the generation phase dominates the inference cost. Each additional token requires a full

forward pass through the model decoder stack. The gap between latency and energy remains tight, indicating that energy scales proportionally with time, as longer generations sustain GPU activity at high utilization. In Figure 3b, throughput increases with output length because longer sequences produce more generated tokens per request. However, the energy cost grows at a comparable rate, resulting in diminishing efficiency improvements. In other words, while raw throughput benefits from longer outputs, the energy per token does not improve significantly. Increasing the batch size (1-8) reduces both latency and energy consumption (Figure 3c). The sharpest drop occurs when moving from batch size 1 to 2, reflecting the substantial efficiency gains from amortizing prefill and kernel launch overhead across multiple concurrent requests. Beyond this point, the reductions continue but with progressively smaller gains. Importantly, the energy profile improves consistently with larger batches, demonstrating that batching is one of the most effective levers to reduce per-workflow energy. As depicted in Figure 3d, throughput increases steadily with larger batches, while energy consumption drops. So, batching offers twofold benefits: it increases the number of tokens processed per unit of energy while reducing the total energy expenditure.

Figure 3e shows that increasing the GPU power cap from 150W to 250W (system default) yields a moderate increase in total energy and does not monotonically reduce latency. At the lowest cap (150W), the system operates at significantly reduced energy levels, but latency penalties become noticeable. As the cap is raised toward 200W, latency improves while energy remains relatively controlled, suggesting an efficiency sweet spot. Increasing the GPU power cap beyond approximately 200W does not reduce latency and instead leads to a slight increase. This behavior indicates that the LLM-powered search workload is no longer power-limited in this regime, as both prefill and decode phases already operate near sustained high GPU utilization. Once the critical path is saturated, additional power does not translate into shorter execution time. Instead, we suspect that latency variations in this regime are dominated by secondary system-level effects such as dynamic voltage and frequency scaling (DVFS)² state transitions and transient scheduling noise³ introduced by continuous batching. Consequently, higher power caps primarily increase instantaneous power draw without improving end-to-end latency, explaining the non-monotonic latency trend at higher power levels. As seen in Figure 3f, throughput increases with power cap but at a roughly proportional increase in energy, suggesting limited benefit beyond moderate cap settings. Overall, this suggests an optimal region (~200W) for balancing efficiency and responsiveness.

Insight: We observe clear performance–energy patterns across the three knobs. Batch size is the most impactful, giving strong throughput gains and lower total energy. Output length increases energy use almost linearly, while power capping works mainly as a fine-tuning control under power limits. Overall, there is a potential for hierarchical energy optimization scheme prioritizing batch size optimization for a given workload following power capping assisted fine-tuning energy usage. Unlike single-request inference studies where these energy knobs show uniform, monotonic improvements in efficiency [58, 64], our LLM-powered search results reveal that shared-prompt reuse amplifies batching gains. This indicates that the same knobs behave in different magnitude once cross-request context sharing and concurrency are introduced, reshaping the efficiency-latency trade-off beyond single-request settings.

5.0.2 RQ2: Cross-workload effects of batch size. To evaluate how the most impactful energy knob (batch size, as identified in RQ1) influences multi-request workloads, we sweep the batch size

² DVFS refers to hardware mechanisms that dynamically adjust processor voltage and clock frequency to balance performance and power consumption.

³ Transient scheduling noise refers to short-term variability in execution timing caused by runtime scheduling decisions, such as request admission, batching, and kernel dispatch, that introduce non-deterministic delays.

parameter for all four workloads introduced in Section 4.2. We fix concurrency at 16 users per workload to ensure comparability and use workload-specific output lengths (maximum completion tokens) consistent with real deployments. All experiments are conducted on *vLLM* with prefix caching enabled. Figure 4a presents the energy per token (J/Token) for all workloads across batch sizes 1–16 (as feasible on our NVIDIA A100-40GB GPU). Here, we plot the mean across repeated runs; the bands show 95% confidence intervals capturing run-to-run variability (we use the same run budget and aggregation method as in Section 5.0.1). If a configuration leads to GPU out-of-memory (OOM) due to the massive KV cache stored, we record it as infeasible and do not report a measured point.

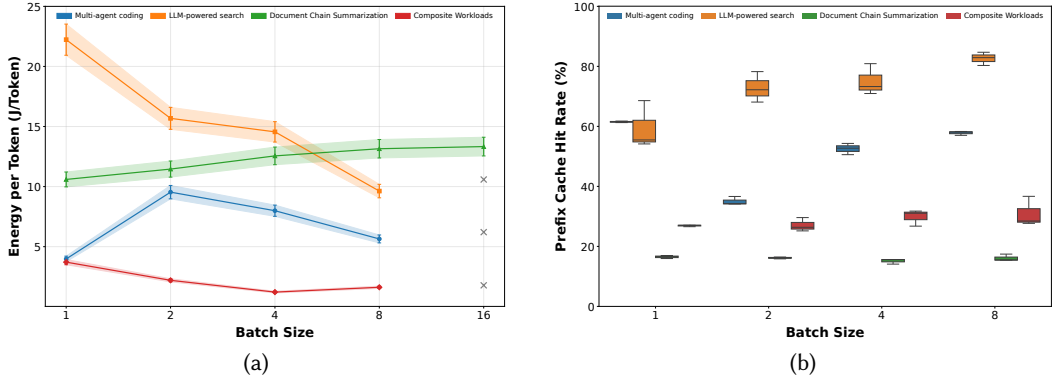


Fig. 4. *a)* Energy per token vs. batch size under 16 concurrent users. Lines show mean value; shaded regions are 95% confidence intervals. Lower is better. The X markers denote batch sizes that are infeasible (OOM). *b)* Prefix cache hit rate distribution across workloads and batch sizes (1–8). Boxplots show medians and variability.

In document chain summarization workload, energy per token increases monotonically with batch size, showing that this decode-heavy workload does not benefit from batching. In contrast, LLM-powered search exhibits the opposite pattern where energy per token drops significantly as batch size increases, reaching its minimum at batch size 8 (corresponding to a $2.6\times$ improvement in efficiency). This outcome is explained by the large shared system prompt (approximately 3000 tokens shared across all requests) which enables high prefix-cache reuse across concurrent queries. Multi-agent coding shows more complex behavior as the lowest energy per token occurs at batch size 1, with efficiency degrading at larger batches but partially recovering at batch size 8. Notably, batch size 2 appears as an outlier with high energy per token. This non-monotonic behavior is primarily driven by disproportionate execution characteristics across agent roles and their interaction with request scheduling in the multi-agent DAG, rather than from measurement noise or run-to-run variance. At batch size 2, the serving system frequently co-batches requests originating from different agent roles (*e.g.*, architect, engineer, reviewer), whose computational profiles differ substantially in terms of prompt length, decode intensity, and runtime. These roles use structurally distinct prompt templates with minimal shared prefixes, leading to poor KV-cache reuse during the prefill stage. As a result, the system incurs batching-related overheads such as queuing delay, GPU residency, and kernel launch costs, without achieving the amortization benefits typically associated with batching. This imbalance inflates GPU energy consumption while resulting in little or no throughput improvement, manifesting as a sharp increase in energy per token. This effect diminishes at larger batch sizes (*e.g.*, batch size 8), where requests from homogeneous roles are more likely to be grouped together. Such batching improves prefix alignment and cache reuse, allowing

overheads to be amortized more effectively across requests. Consequently, the energy efficiency partially recovers at higher batch sizes despite increased concurrency. This behavior highlights that, in multi-agent workflows, batching efficiency depends not only on batch size but also on the semantic and structural alignment of co-batched requests. Finally, for the composite workload, efficiency improves up to batch size 4 due to moderate reuse, but shows reduced marginal benefits thereafter. Overall, sequential dependencies in summarization, role heterogeneity in coding, and mixed patterns in the composite workload restrict batching benefits, while LLM-powered search consistently gains from shared-prefix caching.

To further support this analysis, we leverage *vLLM*'s prefix cache hit rate, defined as the fraction of prefill tokens whose key-value states are reused from prior requests rather than recomputed. A higher hit rate directly translates to reduced redundant computation, lower latency, and improved energy efficiency. Figure 4b shows the distribution of prefix cache hit rates across batch sizes, excluding batch size 16, which is infeasible for most workloads due to GPU memory limits. The results reveal that LLM-powered search achieves both high hit rates and high variability across runs. This variability arises because, although all queries share the same static prompt, appended user queries differ in length and structure. When batch formation aligns queries with similar token lengths, reuse is maximized; when lengths diverge, reuse drops, producing variability. Multi-agent coding maintains moderate and stable hit rates (~45-60%) across batch sizes, reflecting partial reuse from recurring role templates (e.g., coder and reviewer prompts) despite structural diversity among agents. The composite workload exhibits intermediate hit rates (~25-35%) due to the interplay between LLM-powered search and summarization phases where search queries enable reuse, summarization stages contribute less shared context. In contrast, document chain summarization shows consistently low hit rates, as its sequential dependencies and evolving prompts limit prefix overlap across requests. Overall, the prefix cache analysis confirms our earlier observations. Workloads with large, stable shared prefixes (e.g., search) derive the most benefit from batching, whereas sequential or heterogeneous workflows exhibit constrained opportunities for reuse.

Insight: The impact of batch size on energy efficiency varies notably between single- and multi-request inference. In single-request studies, larger batches almost uniformly improve throughput and reduce per-token energy by reducing computation [17, 45]. In contrast, our multi-request evaluation reveals irregular and workload-specific behavior. While LLM-powered search benefits substantially from prefix reuse (up to 2.6× energy reduction), sequential or heterogeneous workflows such as summarization and multi-agent coding show reverse trends where batching increases latency and energy due to queuing delays and fragmented KV caches. This highlights a key difference from prior work; the efficiency of batching in multi-request workflows is driven not only by GPU occupancy [22], but also by cross-request dependency structure and cache reuse dynamics.

5.0.3 RQ3: Comparisons of state-of-the-art serving systems. To answer RQ3, we experiment with the document chain summarization workload, identified in RQ2 as the most energy-intensive application with consistently high energy per token. This workload represents long-form document processing pipelines, where sequential summarization stages accumulate both latency and energy costs. Its decode-heavy nature and limited prefix-cache reuse make it a stringent test case for evaluating the sustainability of serving systems. As part of this evaluation, we randomly selected ten long-form documents from the Arxiv dataset, each containing more than 36,000 tokens. These extensive inputs serve as a realistic stress test for serving systems, as their size far exceeds the context window of the LLM under test and necessitates chunk-based processing.

We perform experiments using both vLLM and Parrot serving systems, systematically varying our key energy knobs. For each knob, we report three evaluation metrics: latency (s), throughput (tokens/s) and total energy (J) broken down across CPU, GPU, and DRAM. To ensure fairness, both serving systems are configured with identical decoding parameters, prompt templates, and measurement intervals.

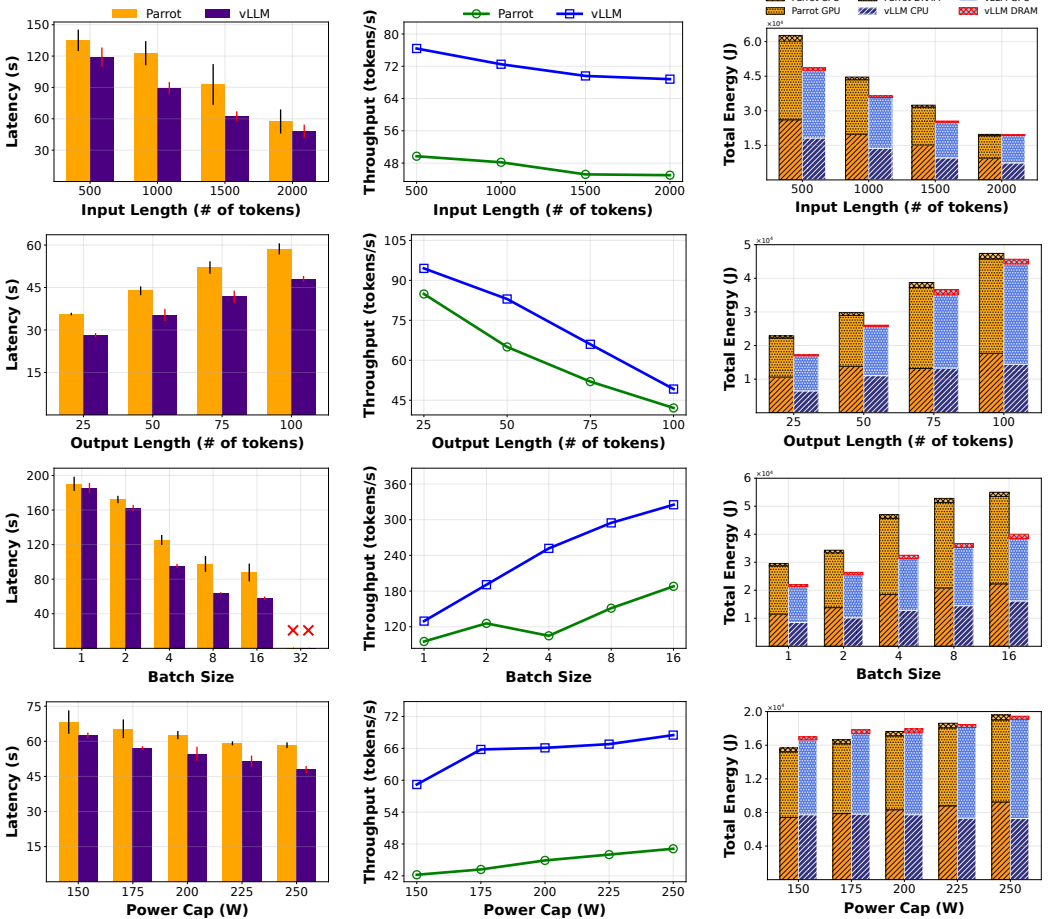


Fig. 5. RQ3 results on chain summarization workload showing impact of input length, output length, batch size, and GPU power capping on latency, throughput, and component-wise (CPU, GPU, and DRAM) energy consumption. The red X markers denote batch sizes that are infeasible due to GPU OOM issue.

Input Length Effect: The input length here corresponds to the token count of each document chunk processed during summarization. In Figure 5, the first row plots the impact of varying input length (500–2000 tokens). Here, latency decreases as larger input length amortize prefill overhead across fewer LLM calls. Throughput remains relatively stable, since prefill scaling does not directly affect decode throughput. Energy consumption decreases significantly with larger input length, as fewer sequential requests are executed. GPU remains the dominant contributor, with CPU and DRAM following similar trends. vLLM achieves consistently lower energy and latency than Parrot, mainly due to more efficient chunked prefill that overlaps computation with decoding.

Output Length Effect: The second row in Figure 5 shows the impact of varying output length (25-100 tokens). Latency and energy rise monotonically as longer generations increase sequential decoding cost, while throughput declines due to extended per-request completion. Unlike the LLM-powered search workload (Figure 3b), where throughput grows with output length for independent requests, sequential dependencies among LLM calls in chain summarization extend the critical path and reduce overall token throughput. vLLM sustains a clear advantage in shorter outputs, consistent with its kernel-level acceleration (see Serving System Comparison).

Batch Size Effect: In Figure 5, the third row plots the impact of increasing batch size. We vary max-num-seqs (maximum concurrent requests per iteration) while holding max-num-batched-tokens fixed to isolate request-level batching effects. Larger batches improve throughput and reduce latency until GPU memory limits are reached, but total energy rises due to heavier GPU utilization. vLLM maintains higher throughput and lower latency across all feasible batch sizes through continuous batching. At batch size 16 (the most energy-intensive configuration among all our experiments), vLLM reduces total energy consumption by up to 28% compared to Parrot. Parrot shows occasional inefficiencies when heterogeneous requests are batched together, reflecting less flexible batching control.

GPU Power Capping Effect: The fourth row in Figure 5 plots the effect of GPU power capping (150W to 250W). Lower power caps moderately reduce total energy but increase latency and reduces throughput. Parrot achieves larger relative energy savings (up to 21% when decreasing the power cap from 250W to 150W) compared to vLLM (12.6%), as its workflow-level scheduling distributes computation more evenly across CPU and GPU under constrained power. However, vLLM sustains higher throughput and lower latency across all caps, reflecting better GPU efficiency under kernel-level optimizations.

Serving System Comparison: The two serving systems embody contrasting optimization philosophies that shape the trends in Figure 5. vLLM prioritizes *engine-level efficiency*, integrating PagedAttention [36], FlashAttention [20], chunked prefill [5] and continuous batching [75] to sustain high GPU utilization and minimize kernel-launch overheads. PagedAttention reduces KV-cache fragmentation by managing memory as paged blocks, while FlashAttention lowers memory traffic during attention computation. Chunked prefill further improves efficiency for long prompts by splitting them into smaller segments that can be processed incrementally and overlapped with decoding, amortizing prefill cost and reducing GPU stall time. Continuous batching complements this by dynamically admitting new requests into active batches, reducing queuing delays. Together, these optimizations collectively enable vLLM to maintain higher and more stable utilization (mean 90.1%, peak 96.0%) compared to Parrot (mean 63.3%, peak 83.0%), as shown in Figure 6. Although vLLM sustains higher GPU utilization, its total energy remains lower because these cycles are efficiently used by reducing idle stalls and shortening latency. The shorter execution time offsets higher instantaneous power, so the overall energy (power $\times$ time) remains lower despite higher utilization.

Parrot, in contrast, adopts a *workflow-aware optimization* paradigm through semantic variables [41], which expose dependency graphs among LLM calls

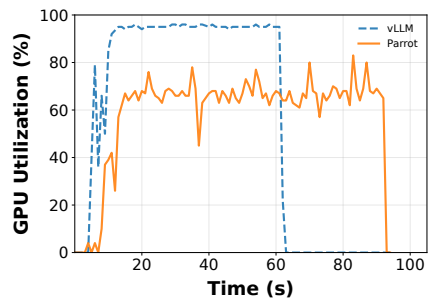


Fig. 6. GPU utilization comparison between vLLM and Parrot for chain summarization workload (batch size 16). Values are averaged across ten runs.

for DAG-level co-scheduling. This approach improves coordination across dependent stages and provides relative gains under aggressive power caps by balancing computation across CPU and GPU resources. However, Parrot does not explicitly mitigate KV-cache fragmentation or optimize fine-grained kernel execution, leading to reduced GPU occupancy at larger batch sizes and higher cumulative energy in sequential workloads (as shown in Figure 5).

Overall, the comparison highlights a broader trade-off between hardware-centric throughput optimization (vLLM) and application-centric workflow scheduling (Parrot). While vLLM’s kernel- and memory-level optimizations provide superior energy efficiency under decode-heavy conditions, Parrot’s DAG-level scheduling achieves modest gains under constrained power budgets, consistent with prior analyses of inference-time efficiency optimizations [22].

Insight: Our comparison between state-of-the-art serving systems vLLM (optimized for engine-level efficiency) and Parrot (optimized for workflow-aware scheduling) identifies a fundamental design tension in multi-request serving. vLLM’s low-level optimizations, including PagedAttention, FlashAttention, and chunked prefill, sustain consistently higher GPU utilization and reduce stall time, resulting in up to 28% lower total energy under decode-heavy, sequential workloads. In contrast, Parrot’s semantic-variable scheduling offers modest benefits only under aggressive power caps, trading GPU saturation for CPU-GPU balance. This result highlights that fine-grained engine-level control over GPU execution often outweighs high-level workflow semantics in determining efficiency. Looking ahead, future serving designs should either (i) prioritize GPU stall elimination to maximize efficiency in decode-bound workflows [42], or (ii) integrate both paradigms in a dynamic, workload-aware manner to combine the complementary strengths of engine-level and workflow-level optimization [66].

6 Discussion

This work establishes a baseline characterization of performance–energy trade-offs in multi-request LLM inference, providing empirical foundations for sustainable system design. Although comprehensive, we acknowledge the following limitations in our study.

First, we restrict our exploration of performance–energy trade-offs to three representative energy-related knobs to isolate their individual effects. Other influential knobs, such as model-level parameters (e.g., quantization, parallelism, activation sparsity) and system-level factors (e.g., multi-GPU scaling, CPU–GPU co-scheduling), are excluded to maintain controlled and reproducible single-GPU evaluations. These excluded knobs interact in complex, often non-linear ways with workload parameters, producing cascading impacts on efficiency [56]. In particular, quantization techniques (e.g., FP16, INT8, or lower-precision formats) are widely used in practice to reduce memory bandwidth pressure and per-token energy consumption, especially during the decode phase. However, quantization may also introduce secondary effects such as increased CPU-side overhead, altered batching behavior, reduced KV-cache efficiency, and, in some cases, longer per-token decode latency depending on kernel maturity and hardware support. While we justify our focus on the three most critical and widely exposed controls in Section 2, future work will extend this characterization to joint tuning of model-, system-, and workload-level knobs, including quantization-aware inference, under distributed and heterogeneous settings.

Second, we restrict our study to Llama 2-7B because it represents a widely used open-source, decoder-only baseline with transparent licensing and stable inference behavior, which is an essential requirement for controlled, reproducible energy measurements. Parrot (at the time of experimentation) does not yet support grouped-query attention (GQA) [6] or mixture-of-experts (MoE) [62] routing, which are core to newer families such as Llama 3, Qwen 3, and DeepSeek R1. Therefore,

evaluating such modern architectures along with large reasoning models (LRMs) remain an important future direction. We plan to incorporate complementary serving systems (e.g., Hugging Face TGI, SGLang) that natively support these architectures to extend our comparative analysis beyond vLLM and Parrot.

Finally, our evaluation is conducted on a single NVIDIA A100 GPU to ensure controlled, repeatable, and fine-grained energy characterization. This setup enables precise attribution of component-level energy without interference from distributed communication overheads. While this single-node design isolates system-level effects, we acknowledge that production LLM inference typically operates on multi-GPU and multi-node clusters. In such environments, interconnect communication over NVLink, PCIe, or InfiniBand, as well as cross-device synchronization, model-parallel coordination, and distributed scheduling, can contribute non-trivial energy overheads that are not captured in our measurements. Extending this study to distributed environments and newer accelerators (e.g., NVIDIA H100, RTX A6000) would enable characterization of these additional energy dynamics and reveal how communication and computation jointly shape end-to-end efficiency under realistic deployment settings.

7 Conclusion

This paper presents a comprehensive assessment of performance–energy trade-offs in multi-request LLM workflows. By deploying four representative workloads and systematically varying key energy knobs, we expose how workload- and system-level decisions shape both efficiency and responsiveness. For the chosen serving systems, vLLM and Parrot, we find that engine-level optimizations like continuous batching and PagedAttention offer significant gains in GPU utilization and energy efficiency, while workflow-aware scheduling provides relative benefits under constrained power budgets. Some of the key insights include, batch size is the most influential knob, but its effects are highly workload-dependent. The output length induces predictable, near-linear energy scaling, offering limited efficiency improvements. GPU power capping can provide modest, controllable savings, but aggressive limits amplify latency penalties. Looking ahead, dynamically adjusting GPU power caps through fine-grained DVFS and automating the tuning of model-, workload-, and system-level energy knobs using reinforcement learning-based approaches represent promising directions toward adaptive, energy-aware LLM serving.

8 Acknowledgments

The authors would like to thank the anonymous SIGMETRICS reviewers and our shepherd for their insightful comments and constructive suggestions, which significantly improved the quality of this paper. We are grateful to Nael Abu-Ghazaleh (Professor at UC Riverside) for valuable feedback and for reviewing final version of the manuscript and to Mosharaf Chowdhury (Associate Professor at University of Michigan) for insightful discussions and contributions to the initial idea development. We also thank Gabriel Spadon De Souza (Assistant Professor at Dalhousie University) for generously providing GPU resources used in this study. This research utilized computational resources from the PINet Lab and the MAPS Labs at Dalhousie University.

References

- [1] 2025. FastAPI. <https://fastapi.tiangolo.com/>.
- [2] 2025. LangChain. <https://github.com/langchain-ai/langchain>.
- [3] 2025. LlamaIndex - Build Knowledge Assistants over your Enterprise Data. <https://www.llamaindex.ai/>.
- [4] Amey Agrawal, Nitin Kedia, Anmol Agarwal, Jayashree Mohan, Nipun Kwatra, Souvik Kundu, Ramachandran Ramjee, and Alexey Tumanov. 2025. On Evaluating Performance of LLM Inference Serving Systems. *arXiv preprint arXiv:2507.09019* (2025).

- [5] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 117–134.
- [6] Joshua Ainslie, James Lee-Thorp, Michiel De Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245* (2023).
- [7] Keivan Alizadeh, Seyed Iman Mirzadeh, Dmitry Belenko, S Khatamifard, Minsik Cho, Carlo C Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. 2024. Llm in a flash: Efficient large language model inference with limited memory. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 12562–12584.
- [8] AMD. 2025. *AMD SMI 26.0.2 documentation*. <https://rocm.docs.amd.com/projects/amdsmi/en/latest/>
- [9] Mauricio Fadel Argerich and Marta Patiño-Martínez. 2024. Measuring and improving the energy efficiency of large language models inference. *IEEE Access* 12 (2024), 80194–80207.
- [10] AutoGen. 2025. *A programming framework for agentic AI*. <https://github.com/microsoft/autogen>
- [11] Hao Bai, Yifei Zhou, Jiayi Pan, Mert Cemri, Alane Suhr, Sergey Levine, and Aviral Kumar. 2024. Digirl: Training in-the-wild device-control agents with autonomous reinforcement learning. *Advances in Neural Information Processing Systems* 37 (2024), 12461–12495.
- [12] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V Le, Christopher Ré, and Azalia Mirhoseini. 2024. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787* (2024).
- [13] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [14] Gohar Irfan Chaudhry, Esha Choukse, Haoran Qiu, Ñigo Goiri, Rodrigo Fonseca, Adam Belay, and Ricardo Bianchini. 2025. Murakkab: Resource-Efficient Agentic Workflow Orchestration in Cloud Platforms. *arXiv preprint arXiv:2508.18298* (2025).
- [15] Jinyuan Chen, Jiuchen Shi, Quan Chen, and Minyi Guo. 2025. Kairos: Low-latency Multi-Agent Serving with Shared LLMs and Excessive Loads in the Public Cloud. *arXiv preprint arXiv:2508.06948* (2025).
- [16] Lingjiao Chen, Jared Quincy Davis, Boris Hanin, Peter Bailis, Ion Stoica, Matei A Zaharia, and James Y Zou. 2024. Are more llm calls all you need? towards the scaling properties of compound ai systems. *Advances in Neural Information Processing Systems* 37 (2024), 45767–45790.
- [17] Krishna Teja Chitty-Venkata, Siddhisanket Raskar, Bharat Kale, Farah Ferdaus, Aditya Tanikanti, Ken Raffanetti, Valerie Taylor, Murali Emani, and Venkatram Vishwanath. 2024. Llm-inference-bench: Inference benchmarking of large language models on ai accelerators. In *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1362–1379.
- [18] Jack Choquette, Wishwesh Gandhi, Olivier Giroux, Nick Stam, and Ronny Krashinsky. 2021. Nvidia a100 tensor core gpu: Performance and innovation. *IEEE Micro* 41, 2 (2021), 29–35.
- [19] Benoit Courty, Victor Schmidt, Sasha Luccioni, Goyal-Kamal, MarionCoutarel, Boris Feld, Jérémy Lecourt, LiamConnell, Amine Saboni, Inimaz, supatomic, Mathilde Léval, Luis Blanche, Alexis Cruveiller, ouminasara, Franklin Zhao, Aditya Joshi, Alexis Bogroff, Hugues de Lavoreille, Niko Laskaris, Edoardo Abati, Douglas Blank, Ziyao Wang, Armin Catovic, Marc Alencon, Michał Stęchły, Christian Bauer, Lucas Otávio N. de Araújo, JPW, and MinervaBooks. 2024. *mlco2/codecarbon: v2.4.1*. doi:10.5281/zenodo.11171501
- [20] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems* 35 (2022), 16344–16359.
- [21] Howard David, Eugene Gorbatoov, Ulf R Hanebutte, Rahul Khanna, and Christian Le. 2010. RAPL: Memory power estimation and capping. In *Proceedings of the 16th ACM/IEEE international symposium on Low power electronics and design*. 189–194.
- [22] Jared Fernandez, Clara Na, Vashisth Tiwari, Yonatan Bisk, Sasha Luccioni, and Emma Strubell. 2025. Energy Considerations of Large Language Model Inference and Efficiency Optimizations. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 32556–32569. doi:10.18653/v1/2025.acl-long.1563
- [23] Eva García-Martín, Crefeda Faviola Rodrigues, Graham Riley, and Håkan Grahñ. 2019. Estimation of energy consumption in machine learning. *J. Parallel and Distrib. Comput.* 134 (2019), 75–88.
- [24] Stefanos Georgiou, Maria Kechagia, Tushar Sharma, Federica Sarro, and Ying Zou. 2022. Green ai: Do deep learning frameworks have different costs?. In *Proceedings of the 44th International Conference on Software Engineering*. 1082–1094.
- [25] ggml-org. 2025. *ggml-org/llama.cpp: LLM inference in C/C++*. <https://github.com/ggml-org/llama.cpp>
- [26] Google. 2025. *Gemma open models*. <https://ai.google.dev/>

- [27] Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi, Heyang Qin, Arash Bakhtiari, Lev Kurilenko, et al. 2024. Deepspeed-fastgen: High-throughput text generation for llms via mii and deepspeed-inference. *arXiv preprint arXiv:2401.08671* (2024).
- [28] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xianwu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. 2023. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*.
- [29] Hongzhen Huang, Kunming Zhang, Hanlong Liao, Kui Wu, and Guoming Tang. 2025. WattsOnAI: Measuring, Analyzing, and Visualizing Energy and Carbon Footprint of AI Workloads. *arXiv preprint arXiv:2506.20535* (2025).
- [30] huggingface. 2025. *huggingface/text-generation-inference: Large Language Model Text Generation Inference*. <https://github.com/huggingface/text-generation-inference>
- [31] HuggingFace. 2025. *meta-llama/Llama-2-7b*. <https://huggingface.co/meta-llama/Llama-2-7b>
- [32] International Energy Agency (IEA). 2024. *Energy Efficiency 2024*. Paris. <https://www.iea.org/reports/energy-efficiency-2024> Licence: CC BY 4.0.
- [33] International Energy Agency (IEA). 2025. *Energy Efficiency 2025*. <https://www.iea.org/reports/energy-and-ai>
- [34] Jie You and Jae-Won Chung and Mosharaf Chowdhury. 2025. *ml-energy/zeus: Measure and optimize the energy consumption of your AI applications!* <https://github.com/ml-energy/zeus>
- [35] Andreas Kosmas Kakolyris, Dimosthenis Masouros, Sotirios Xydis, and Dimitrios Soudris. 2024. Slo-aware gpu dvfs for energy-efficient llm inference serving. *IEEE Computer Architecture Letters* 23, 2 (2024), 150–153.
- [36] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.
- [37] Seyyidahmed Lahmer, Aria Khoshshirat, Michele Rossi, and Andrea Zanella. 2022. Energy consumption of neural networks on nvidia edge boards: an empirical model. In *2022 20th international symposium on modeling and optimization in mobile, ad hoc, and wireless networks (WiOpt)*. IEEE, 365–371.
- [38] Baolin Li, Yankai Jiang, Vijay Gadepally, and Devesh Tiwari. 2024. Llm inference serving: Survey of recent advances and opportunities. In *2024 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 1–8.
- [39] Yucheng Li. 2023. Unlocking context constraints of llms: Enhancing context efficiency of llms with self-information-based content filtering. *arXiv preprint arXiv:2304.12102* (2023).
- [40] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E Gonzalez, et al. 2023. {AlpaServe}: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 663–679.
- [41] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient serving of {LLM-based} applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 929–945.
- [42] Zejia Lin, Hongxin Xu, Guanyi Chen, Xianwei Zhang, and Yutong Lu. 2025. Bullet: Boosting GPU Utilization for LLM Serving via Dynamic Spatial-Temporal Orchestration. *arXiv preprint arXiv:2504.19516* (2025).
- [43] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranajape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172* (2023).
- [44] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E Gonzalez, et al. 2025. Autellix: An efficient serving engine for llm agents as general programs. *arXiv preprint arXiv:2502.13965* (2025).
- [45] Paul Joe Maliakel, Shashikant Ilager, and Ivona Brandic. 2025. Investigating Energy Efficiency and Performance Trade-offs in LLM Inference Across Tasks and DVFS Settings. *arXiv preprint arXiv:2501.08219* (2025).
- [46] MetaGPT. 2025. *The Multi-Agent Framework*. <https://github.com/FoundationAgents/MetaGPT>
- [47] Microsoft. 2025. *Microsoft 365 Copilot*. <https://www.microsoft.com/en-us/microsoft-365/enterprise/microsoft-365-copilot>
- [48] Microsoft. 2025. *microsoft/ParrotServe: [OSDI'24] Serving LLM-based Applications Efficiently with Semantic Variable*. <https://github.com/microsoft/ParrotServe/tree/main>
- [49] Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. 2024. Large language models: A survey. *arXiv preprint arXiv:2402.06196* (2024).
- [50] NVIDIA Developer. 2025. *System Management Interface SMI*. <https://developer.nvidia.com/system-management-interface>
- [51] Ollama. 2025. *Get up and running with OpenAI gpt-oss, DeepSeek-R1, Gemma 3 and other models*. <https://github.com/ollama/ollama>
- [52] Open Collective. 2025. *vllm-project/vllm: A high-throughput and memory-efficient inference and serving engine for LLMs*. <https://github.com/vllm-project/vllm>
- [53] Open Collective. 2025. *vLLM V1 User Guide — vLLM*. https://docs.vllm.ai/en/v0.8.1/getting_started/v1_user_guide.html

- [54] Pratyush Patel, Esha Choukse, Chaojie Zhang, Íñigo Goiri, Brijesh Warriar, Nithish Mahalingam, and Ricardo Bianchini. 2024. Characterizing power management opportunities for llms in the cloud. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 207–222.
- [55] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 118–132.
- [56] Saurabhsingh Rajput, Mootez Saad, and Tushar Sharma. 2025. Tu (r) ning AI Green: Exploring Energy Efficiency Cascading with Orthogonal Optimizations. *arXiv preprint arXiv:2506.18289* (2025).
- [57] Saurabhsingh Rajput, Tim Widmayer, Ziyuan Shang, Maria Kechagia, Federica Sarro, and Tushar Sharma. 2024. Enhancing energy-awareness in deep learning through fine-grained energy measurement. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–34.
- [58] Siddharth Samsi, Dan Zhao, Joseph McDonald, Baolin Li, Adam Michaleas, Michael Jones, William Bergeron, Jeremy Kepner, Devesh Tiwari, and Vijay Gadepally. 2023. From words to watts: Benchmarking the energy costs of large language model inference. In *2023 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 1–9.
- [59] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems* 36 (2023), 68539–68551.
- [60] sglang.ai. 2025. *sgl-project/sglang: SGLang is a fast serving framework for large language models and vision language models*. <https://github.com/sgl-project/sglang>
- [61] ShareGPT team. 2025. *Sharegpt dataset*. <https://sharegpt.com/>
- [62] N Shazeer, A Mirhoseini, K Maziarz, A Davis, Q Le, G Hinton, and J Dean. 2017. The sparsely-gated mixture-of-experts layer. *Outrageously large neural networks 2* (2017).
- [63] Ghazal Sobhani, Md Monzurul Amin Ifath, Tushar Sharma, and Israat Haque. 2025. On the Sustainability of AI Inferences in the Edge. *arXiv preprint arXiv:2507.23093* (2025).
- [64] Jovan Stojkovic, Esha Choukse, Chaojie Zhang, Íñigo Goiri, and Josep Torrellas. 2024. Towards Greener LLMs: Bringing Energy-Efficiency to the Forefront of LLM Inference. In *EMC2 at ASPLOS*.
- [65] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2025. Dynamollm: Designing llm inference clusters for performance and energy efficiency. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 1348–1362.
- [66] Zhaoyuan Su, Tingfeng Lan, Zirui Wang, Juncheng Yang, and Yue Cheng. 2025. Efficient and Workload-Aware LLM Serving via Runtime Layer Swapping and KV Cache Resizing. *arXiv preprint arXiv:2506.02006* (2025).
- [67] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: Dynamic scheduling for large language model serving. In *18th USENIX symposium on operating systems design and implementation (OSDI 24)*. 173–191.
- [68] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [69] Xiaolong Tu, Anik Mallik, Dawei Chen, Kyungtae Han, Onur Altintas, Haoxin Wang, and Jiang Xie. 2023. Unveiling energy efficiency in deep learning: Measurement, prediction, and scoring across edge devices. In *Proceedings of the Eighth ACM/IEEE Symposium on Edge Computing*. 80–93.
- [70] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [71] Grant Wilkins, Srinivasan Keshav, and Richard Mortier. 2024. Offline energy-optimal llm serving: Workload-based energy models for llm inference on heterogeneous systems. *ACM SIGENERGY Energy Informatics Review* 4, 5 (2024), 113–119.
- [72] Carole-Jean Wu, Ramya Raghavendra, Udit Gupta, Bilge Acun, Newsha Ardalani, Kiwan Maeng, Gloria Chang, Fiona Aga, Jinshi Huang, Charles Bai, et al. 2022. Sustainable ai: Environmental implications, challenges and opportunities. *Proceedings of machine learning and systems* 4 (2022), 795–813.
- [73] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. 2023. Autogen: Enabling next-gen llm applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155* 3, 4 (2023).
- [74] Jie You, Jae-Won Chung, and Mosharaf Chowdhury. 2023. Zeus: Understanding and optimizing {GPU} energy consumption of {DNN} training. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 119–139.
- [75] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 521–538.

- [76] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems* 36 (2023), 46595–46623.
- [77] Xuanhe Zhou, Xinyang Zhao, and Guoliang Li. 2024. Llm-enhanced data management. *arXiv preprint arXiv:2402.02643* (2024).
- [78] Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. 2024. Language agents as optimizable graphs. *arXiv preprint arXiv:2402.16823* (2024).

Received October 2025; revised December 2025; accepted January 2026